

# Backend con Node.js

Guía de Conceptos Detallados - Unidad 1

## 1. ¿Qué es Node.js?

Node.js es un entorno de ejecución para JavaScript construido con el motor V8 de Google Chrome. Permite ejecutar código JavaScript en el servidor, fuera del entorno tradicional del navegador web.

**Definición Oficial:** "Node.js es un entorno de ejecución para JavaScript orientado a eventos asíncronos, diseñado para construir aplicaciones en red escalables."

A diferencia de lenguajes tradicionales que operan de forma sincrónica y bloqueante, Node.js se destaca por ser ligero y eficiente, ideal para aplicaciones en tiempo real con datos intensivos.

## 2. Arquitectura y Características Principales

- **Asíncrono y orientado a eventos:** Las APIs de Node.js son no bloqueantes. El sistema no espera a que una operación (como leer un archivo o consultar una base de datos) termine para continuar ejecutando el resto del código.
- **Rápido:** Al estar impulsado por el motor V8 de Chrome, la compilación y ejecución de JavaScript a código máquina es extremadamente rápida.
- **Single thread pero altamente escalable:** Utiliza un solo hilo (single thread) principal combinado con un "bucle de eventos" (Event Loop) para manejar múltiples conexiones concurrentes sin colapsar el servidor.

## 3. Modelo de E/S (Input/Output) Sin Bloqueo

Para entender el poder de Node.js, es vital comprender la diferencia entre modelos bloqueantes y no bloqueantes:

| Modelo Síncrono (Bloqueante)                                          | Modelo Asíncrono (No Bloqueante - Node.js)                                          |
|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| El programa lee un archivo y se detiene completamente hasta terminar. | El programa inicia la lectura del archivo y delega la tarea.                        |
| Las siguientes líneas de código deben esperar.                        | El código subsiguiente se ejecuta inmediatamente.                                   |
| Consume más recursos en esperas pasivas.                              | Cuando el archivo está listo, se ejecuta una función (Callback) con los resultados. |

## 4. Sistema de Módulos y NPM

Node.js utiliza el sistema CommonJS para organizar y reutilizar el código. Los módulos encapsulan funcionalidades para mantener el código limpio y seguro.

### Tipos de Módulos

- Módulos Core:** Vienen integrados en Node.js.
  - `fs` (File System): Para interactuar con archivos (leer, escribir, borrar).
  - `path`: Para manejar rutas de directorios de manera segura en cualquier sistema operativo.
  - `http` / `https`: Para levantar servidores web y manejar peticiones de red.
- Módulos Locales:** Creados por el desarrollador dentro del proyecto.
- Módulos de Terceros:** Instalados mediante **npm** (Node Package Manager).

**NPM y package.json:** npm es el gestor de paquetes que permite instalar bibliotecas externas. Al iniciar un proyecto (`npm init`), se crea el archivo `package.json`, el cual funciona como el "documento de identidad" del proyecto, guardando los metadatos, scripts de ejecución y todas las dependencias necesarias.

## 5. El Manejo de la Asincronía

Node.js ofrece varias formas de manejar el código que toma tiempo en ejecutarse:

- **Callbacks:** La forma clásica. Es una función que se pasa como argumento a otra función y se ejecuta cuando la tarea termina. Si se anidan muchos, generan el temido "Callback Hell".
- **Promesas:** Objetos que representan el éxito o fracaso eventual de una operación asíncrona (usando `.then()` y `.catch()`).
- **Async/Await:** La sintaxis más moderna (introducida en ES2017). Permite escribir código asíncrono que se lee como si fuera código síncrono, utilizando bloques `try/catch` para un manejo de errores limpio.

## 6. El Modelo de Eventos (EventEmitter)

Gran parte de la arquitectura interna de Node.js está basada en eventos. A través de la clase `EventEmitter` del módulo `events`, se pueden crear objetos (emisores) que emitan eventos y asignarles funciones (listeners) que reaccionen cuando ese evento ocurra.

## 7. Casos de Uso: ¿Cuándo usar Node.js?

| Ideal Para (Recomendado)                          | No Recomendado Para                                                                                 |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Aplicaciones en tiempo real (Chats, Juegos)       | Aplicaciones con uso intensivo de CPU (Cálculos matemáticos complejos, renderizado de video pesado) |
| API RESTful y Microservicios                      | Aplicaciones monolíticas tradicionales muy grandes                                                  |
| Streaming de audio o video                        |                                                                                                     |
| Aplicaciones Single Page (SPA) y herramientas CLI |                                                                                                     |

## 8. Buenas Prácticas Fundamentales

- Capturar y manejar siempre los errores en las operaciones asíncronas.
- Estructurar el código separando responsabilidades (rutas, controladores, servicios).
- Usar variables de entorno ( `.env` ) para proteger contraseñas y claves de API.
- Evitar el "Callback Hell" priorizando el uso de `Async/Await`.
- Validar estrictamente todos los datos de entrada provenientes del usuario.